

Create Custom Wordlist Using Crunch

By Tinku - April 29, 2021

```
root@kali:~# cat /usr/share/rainbowcrack/charset.txt

numeric          = [0123456789]
alpha            = [ABCDEFGHIJKLMNOPQRSTUVWXYZ]
alpha-numeric    = [ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789]
loweralpha       = [abcdefghijklmnopqrstuvwxyz]
loweralpha-numeric = [abcdefghijklmnopqrstuvwxyz0123456789]
mixalpha         = [abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ]
mixalpha-numeric = [abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789]

ascii-32-95      = [ !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMN
OPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~]
ascii-32-65-123-4 = [ /!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMN
OPQRSTUVWXYZ[\]^_`{|}~]
alpha-numeric-symbol32-space = [ ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%^&*()_-
+=~`[]{}|\:;'"<>.,?/ ]

oracle-alpha-numeric-symbol3 = [ ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789#$% ]
```

Create Custom Wordlist Using Crunch

Wordlist

Wordlist is the collection of words (Numbers, Alphabets, Special Characters etc). Wordlist help to crack password easily as compare to Brute Force attack.

Crunch

Crunch is a Built-in tool or Pre-installed tool in Kali Linux. Crunch is a wordlist generator tool where you can specify a standard character set or a character set you specify. crunch can generate all possible combinations and permutations. We can create custom wordlist using Crunch tool.

Create Custom Wordlist by Using Crunch

If you simply type 'crunch' in your Kali Linux terminal, you will see this type of screen

Crunch Syntax

```
# crunch <min><max><character set> -t <pattern> -o <output filename>
```

```
kali@kali:~$ crunch
crunch version 1.6

Crunch can create a wordlist based on criteria you specify. The output from crunch can be sent to the screen, file, or to another program.

Usage: crunch min max [options]
where min and max are numbers

Please refer to the man page for instructions and examples on how to use crunch.
```

If I normally type

```
# crunch 4 6
```

It will create Large wordlist because we not indicate any character that crunch tool make the wordlist. Where 4 is minimum length of password and 5 is maximum length of password,

```
kali@kali:~$ crunch 4 6
Crunch will now generate the following amount of data: 2235983568 bytes
2132 MB
2 GB
0 TB
0 PB
Crunch will now generate the following number of lines: 321254128
```

It will show like this,

```
frkjd  
frkje  
frkjf  
frkjg  
frkjh  
frkji  
frkjj  
frkjk  
frkjl  
frkjm  
frkjn  
frkjo  
frkjp  
frkjq  
frkjr  
frkjs  
frkjt  
frkju  
frkjb  
frkjc  
frkjd
```

Now we define, where is my output of wordlist generated.

```
# crunch 4 6 -o /root/
```

```
kali@kali:~$ crunch 4 6  
Crunch will now generate the following amount of data: 2235983568 bytes  
2132 MB  
2 GB  
0 TB  
0 PB  
Crunch will now generate the following number of lines: 321254128
```

Crunch Other Options

We check manual entry of Crunch tool, by type

```
# man crunch
```

You will see this type of screen

- b : the maximum size of the wordlist (requires -o START)
- c : numbers of lines to write to the wordlist (requires -o START)
- d : limit the number of duplicate characters
- e : stop generating words at a certain string
- f : specify a list of character sets from the charset.lst file
- i : invert the order of characters in the wordlist
- l : allows the literal interpretation of @,%^ when using -t
- o : the output wordlist file
- p : print permutations without repeating characters (cannot be used with -s)
- q : Like the -p option except it reads the strings from a specified file
- r : resume a previous session (cannot be used with -s)
- s : specify a particular string to begin the wordlist with
- t : set a specific pattern of @,%^
- z : compress the output wordlist file, accompanied by -o

Where,

@ represents lowercase letters

, represents uppercase letters

% represents numbers

^ represents special characters

Creating WordLists for Penetration Testing with Crunch

 Security and Authentication  nauseous  Tuesday, 24 November 2015  14250 Hits

What is Crunch?

Crunch is a **wordlist generator** where you can specify a standard character set or a character set you specify.

Crunch can **generate** all possible **combinations** and **permutations**. **Crunch** can create a **wordlist** based on **criteria** you specify. The output from **Crunch** can be sent to the **screen**, **file**, or to **another program**.

Features

- **Crunch generates wordlists** in both **combination** and **permutation** ways it can breakup output by number of lines or file size
- Now has **resume support**
- **Pattern** now supports **number and symbols**
- **Pattern** now supports **upper** and **lower case characters separately**
- Adds a **status report** when generating multiple files
- New **-l** option for literal **support** of **@,%^**
- New **-d** option to **limit duplicate characters** see man file for details
- Now has **unicode support**

Note: We use **Crunch** for penetration testing our web servers, wireless AP's and for firewall testing. It's useful and can be used with programs or scripts to safeguard systems.

Install Crunch 3.6 Wordlist Generator

Download **Crunch**:

```
# wget http://downloads.sourceforge.net/project/crunch-wordlist/crunch-wordlist/crunch-3.6.tgz
```

Extract **crunch-3.6.tgz**

```
# tar zxvf crunch-3.6.tgz
```

Output:

```
crunch-3.6/
crunch-3.6/crunch.1
crunch-3.6/charset.lst
crunch-3.6/unicode_test.lst
crunch-3.6/Makefile
crunch-3.6/crunch.c
crunch-3.6/COPYING
```

Change directories to the extracted Crunch

```
# cd crunch-3.6/
```

Now we can make Crunch

```
# make
```

Output:

```
Building binary...
/usr/bin/gcc -pthread -Wall -pedantic -std=c99 crunch.c -lm -o crunch
```

Now we can install the source

```
# make install
```

Creating directories...

```
sudo /usr/bin/install -d -g root -o root \
  /usr/bin \
  /usr/share/man/man1 \
  /usr/share/Crunch\
  /usr/share/doc/crunch
```

Copying binary...

```
sudo /usr/bin/install Crunch/usr/bin
```

Copying charset.lst...

```
sudo /usr/bin/install charset.lst /usr/share/crunch
```

Copying COPYING...

```
sudo /usr/bin/install COPYING /usr/share/doc/crunch
```

Installing man page...

```
sudo /usr/bin/install crunch.1 /usr/share/man/man1
```

Now we can type **Crunch** and get:

```
# crunch
```

Default Output without parameters:

Crunch version 3.6

Crunch can create a wordlist based on criteria you specify. The output from **Crunch** can be sent to the screen, file, or to another program.

Usage: **Crunch** [options]

where min and max are numbers

Please refer to the man page for instructions and examples on how to use crunch.

Now we can create some passwords with **Crunch**

Syntax of Crunch:

Crunch[min length] [max length] [character set] [options]

The below command will instruct **Crunch** to create a wordlist that will have minimum length of characters 5, maximum length of characters 5 with the characters of abcdefghijklmnopqrstuvwxyz and it will save it as a .txt file called atoz-pass.txt.

```
# Crunch 5 5 abcdefghijklmnopqrstuvwxyz -o atoz-pass.txt
```

Output:

Crunch will now generate the following amount of data: 71288256 bytes

67 MB

0 GB

0 TB

0 PB

Crunch will now generate the following number of lines: 11881376

crunch: 100% completed generating output

Output of file is:

aqvom

aqvon

aqvoo

aqvop
aqvoq
aqvor
aqvos
aqvot
...

It only took about 10 seconds to create a list of 11881376 lines and around 67MB of passwords.

You can also use numbers to create a combination of passwords like below example.

```
# Crunch 8 8 1234567890 -o 1to0-8chars.txt
```

You can also create wordlists with letters and numbers like below.

```
# Crunch 5 5 pentest12345 -o let-num.txt
```

As well as creating special characters like:

```
# Crunch 8 8 pentest\@)\#\$\%\^\&\! -o textnum.txt
```

The above command will create a 3690MB which is a 3GB file called textnum.txt.

Now let's generate a wordlist with 8 characters, 2mb each using numbers and letters

```
# Crunch 8 8 123linux -b 2mb -o START
```

Output:

Crunch will now generate the following amount of data: 150994944 bytes

144 MB

0 GB

0 TB

0 PB

Crunch will now generate the following number of lines: 16777216

crunch: 1% completed generating output

crunch: 2% completed generating output

crunch: 3% completed generating output

crunch: 4% completed generating output

Crunch also allows a maximum number of words in a output file like example below. **Crunch** will create a maximum of 15 words max with a length of 5 characters per password and minimum password length of 5 characters by using alpha characters -->> "abcdefghijklmnopqrstuvwxyz"

```
# Crunch 5 5 -f charset.lst alpha -o START -c 15
```

Creating Prefix WordLists

Specifies a pattern, eg: @linux@@@@ where the only the @'s, , 's, %'s, and ^'s will change.

- @ will insert lower case characters
- , will insert upper case characters
- % will insert numbers
- ^ will insert symbols

```
# Crunch 8 8 -f charset.lst alpha -t linuxsecrets@@@@ -o linuxsecrets.txt
```

Output example:

linuxsecretsdadg

linuxsecretsdadh

linuxsecretsdadi

linuxsecretsdadj

linuxsecretsdadk

linuxsecretsdadl

linuxsecretsdadm

linuxsecretsdadn

linuxsecretsdado

Crunch also comes with fixed character sets in charset.lst which is included in the installation. So, we can use a charset list (like JTR tutorial) to create a wordlist like the following example:

```
# Crunch 4 4 -f charset.lst lalpha-numeric
```

Output:

Crunch will now generate the following amount of data: 8398080 bytes

8 MB

0 GB

0 TB

0 PB

Crunch will now generate the following number of lines: 1679616

aaaa

aaab

...

sje0

sje1

...

2rii

2rij

2rik

Note: We can use the -i option to invert the direction in which the wordlist is created, from left-to-right to right-to-left

Also using the parameter -o [destination_path] **Crunch** will create an output file to the specific path destination

```
# Crunch 4 4 -f charset.lst lalpha-numeric -o /Desktop/wordlist.txt
```

To compress the output wordlist use -z parameter

```
# Crunch 4 4 -f charset.lst lalpha-numeric -o /Desktop/wordlist.txt -z gzip
```

Creating Wordlist - Advanced Usage

Crunch also supports a lot of advanced option for creating complex wordlists and a couple of parameters which help us for compressing an existing wordlist.

At this part of tutorial will describe the parameter -t. Adding this parameter allows to specify a pattern or a pre-fix at the wordlist. Note that using special character without escaping it **Crunch** will automatically generate different wordlist for each one. For example:

```
# Crunch 5 5 -t wl@@@
```

will create a wordlist which will has as prefix the word "wl" and three lower case alpha values.

```
# Crunch 5 5 -t wl"
```

wordlist 'll contain the prefix "wl" and three upper case alpha value.

```
# Crunch 5 5 -t %%%wl
```

wordlist 'll contain three numeric values and the suffix "wl".

```
# Crunch 5 5 -t ^wl^^
```

wordlist 'll contain a special characters, the word "wl" and then two special characters. (including 'space' character)

Note: We can create a wordlist including more than one of the above options.

```
# Crunch 10 10 -t w!,,^^@%%%
```

If would like to create a wordlist with a prefix which prefix will contain a special character, we must use the parameter -l and specify the special character and the location of it in the wordlist. For example:

```
# Crunch 6 6 -t '@dm'@@@ -l @aaaa
```

Crunch will now generate the following amount of data: 123032 bytes

0 MB
0 GB
0 TB
0 PB

Crunch output information will now generate the following number of lines: 17576

@dmaaa
@dmaab
@dmaac
@dmaad
@dmaae
@dmaaf
:

Note: that now a special character '@' threated and used as literal. A good tip is to add our prefix word in tabs, to separate it from the special characters.

Edit Charset.txt List

On this part of Tutorial will edit and create our set of chars on charset.txt file:

```
# charset configuration file for winrtgen v1.2 by Massimiliano Montoro (mao@oxid.it)  
# compatible with rainbowcrack 1.1 and later by Zhu Shuanglei <shuanglei@hotmail.com>;;
```

```
hex-lower      = [0123456789abcdef]  
hex-upper      = [0123456789ABCDEF]
```

```
numeric        = [0123456789]  
numeric-space  = [0123456789 ]
```

```
symbols14      = [!@#$$%^&*()-_+=]  
symbols14-space = [!@#$$%^&*()-_+= ]
```

```
symbols-all    = [!@#$$%^&*()-_+=~`[]{}|\;'"<>.,?/]  
symbols-all-space = [!@#$$%^&*()-_+=~`[]{}|\;'"<>.,?/ ]
```

```
ualpha         = [ABCDEFGHJKLMNOPQRSTUVWXYZ]  
ualpha-space   = [ABCDEFGHJKLMNOPQRSTUVWXYZ ]  
ualpha-numeric = [ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789]  
ualpha-numeric-space = [ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789 ]  
ualpha-numeric-symbol14 = [ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789!@#$$%^&*()-_+=]  
ualpha-numeric-symbol14-space = [ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789!@#$$%^&*()-_+= ]  
ualpha-numeric-all = [ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789!@#$$%^&*()-_+=~`[]{}|\;'"<>.,?/]  
ualpha-numeric-all-space = [ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789!@#$$%^&*()-_+=~`[]{}|\;'"<>.,?/ ]  
:
```

We can add our set of character like the following example:

set_char_name = [my,chars,-,symbols]

and can use it adding the name of our set like as an already char set.

```
# Crunch 6 6 -f charset.lst my_charset -t 'p@'@@@ -l a@aaaa -o /Desktop/wordlist.txt
```

Crunch will now generate the following amount of data: 229376 bytes

0 MB
0 GB
0 TB
0 PB

Crunch will now generate the following number of lines: 32768

p@\$AAA
p@\$AAB
p@\$AAC
p@\$AAD
p@\$AAa
p@\$AAb
p@\$AAc
p@\$AAd
:

Other **Crunch** options below:

```
# Crunch 1 8
```

Crunch will display a wordlist that starts at a and ends at zzzzzzzz

```
# Crunch 1 6 abcdefg
```

Crunch will display a wordlist using the character set abcdefg that starts at a and ends at gggggg

```
# Crunch 1 6 abcdefg\
```

There is a space at the end of the character string. In order for **Crunch** to use the space you will need to escape it using the \ character. In this example you could also put quotes around the letters and not need the \, i.e. "abcdefg".

Crunch will display a wordlist using the character set abcdefg that starts at a and ends at (6 spaces)

```
# Crunch 1 8 -f charset.lst mixalpha-numeric-all-space -o wordlist.txt
```

Crunch will use the mixalpha-numeric-all-space character set from charset.lst and will write the wordlist to a file named wordlist.txt. The file will start with a and end with " "

```
# Crunch 8 8 -f charset.lst mixalpha-numeric-all-space -o wordlist.txt -t @.dog@.@. -s cbdogaaa
```

Crunch should generate a 8 character wordlist using the mixalpha-number-all-space character set from charset.lst and will write the wordlist to a file named wordlist.txt. The file will start at cbdogaaa and end at " dog "

```
# Crunch 2 3 -f charset.lst ualpha -s BB
```

Crunch will start generating a wordlist at BB and end with ZZZ. This is useful if you have to stop generating a wordlist in the middle. Just do a tail wordlist.txt and set the -s parameter to the next word in the sequence. Be sure to rename the original wordlist BEFORE you begin as **Crunch** will overwrite the existing wordlist.

```
# Crunch 4 5 -p abc
```

The numbers aren't processed but are needed. **Crunch** will generate abc, acb, bac, bca, cab, cba.

```
# Crunch 4 5 -p dog cat bird
```

The numbers aren't processed but are needed. **Crunch** will generate birdcatdog, birddogcat, catbirddog, catdogbird, dogbirdcat, dogcatbird.

```
# Crunch 1 5 -o START -c 6000 -z bzip2
```


Crunch will generate bzip2 compressed files with each file containing 6000 words. The filenames of the compressed files will be first_word-last_word.txt.bz2

```
# time # Crunch 1 4 -o START -c 6000 -z gzip
```

Output:

```
real 0m2.729s
user 0m2.216s
sys 0m0.360s
```

```
# time # Crunch 1 4 -o START -c 6000 -z bzip2
```

Output:

```
real 0m3.414s
user 0m2.620s
sys 0m0.580s
```

```
# time # Crunch 1 4 -o START -c 6000 -z lzma
```

Output:

```
real 0m43.060s
user 0m9.965s
sys 0m32.634s
```

```
size filename
30K aaaa-aiwt.txt
12K aaaa-aiwt.txt.gz
3.8K aaaa-aiwt.txt.bz2
1.1K aaaa-aiwt.txt.lzma
```

```
# Crunch 4 5 -b 20mib -o START
```

Will generate 4 files: aaaa-gvfd.txt, gvfee-ombqy.txt, ombqz-wcydt.txt, wcydu-zzzzz.txt the first three files are 20MBs (real power of 2 MegaBytes) and the last file is 11MB.

```
# Crunch 3 3 abc + 123 !@# -t @%^
```

Will generate a 3 character long word with a character as the first character, and number as the second character, and a symbol for the third character. The order in which you specify the characters you want is important. You must specify the order as lower case character, upper case character, number, and symbol. If you aren't going to use a particular character set you use a plus sign as a placeholder. As you can see I am not using the upper case character set so I am using the plus sign placeholder.

The above will start at a1! and end at c3#

```
# Crunch 3 3 abc + 123 !@# -t ^%@
```

Will generate 3 character words starting with !1a and ending with #3c

```
# Crunch 4 4 ++ 123 + -t %%@^<br< a="">>
```

The plus sign (+) is a placeholder so you can specify a character set for the character type. **Crunch** will use the default character set for the character type when **Crunch** encounters a + (plus sign) on the command line. You must either specify values for each character type or use the plus sign. I.E. if you have two characters types you MUST either specify values for each type or use a plus sign. So in this example the character sets will be:

```
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
123
!@#$%^&*(-_+=~`[]{}|~;"'<>.,?/
```

There is a space at the end of the above string the output will start at 11a! and end at "33z ". The quotes show the space at the end of the string.

```
# Crunch 5 5 -t ddd@@@ -o j -p dog cat birds
```

Any character other than one of the following: @,%^ is the placeholder for the words to permute. The @,%^ symbols have the same function as -t. If you want to use @,%^ in your output you can use the -l option to specify which character you want **Crunch** to treat as a literal. So the results are birdcatdogaa

birdcatdogab
birdcatdogac

dogcatbirdzy
dogcatbirdzz

```
# Crunch 7 7 -t p@ss,%^ -l a@aaaaa<br< a="">>
```

Crunch will now treat the @ symbol as a literal character and not replace the character with a uppercase letter. This will generate

```
p@ssA0!<br< a="">>    p@ssA0@  
p@ssA0#<br<; a="">>    p@ssA0$<br<; a="">>  
p@ssZ9>;
```

```
# Crunch 5 5 -s @4#S2 -t @%^,2 -e @8 Q2 -l @dddd -b 10KB -o START
```

Crunch will generate 5 character strings starting with @4#S2 and ending at @8 Q2. The output will be broken into 10KB sized files named for the files starting and ending strings.

```
# Crunch 5 5 -d 2@ -t @@@@%%
```

Crunch will generate 5 character strings starting with aab00 and ending at zzy99. Notice that aaa and zzz are not present.

```
# Crunch 10 10 -t @@@^%%%%%%%%^^ -d 2@ -d 3% -b 20mb -o START
```

Crunch will generate 10 character strings starting with aab!0001!! and ending at zzy 9998. The output will be written to 20mb files.

```
# Crunch 8 8 -d 2@
```

Crunch will generate 8 characters that limit the same number of lower case characters to 2. Crunch will start at aabaabaa and end at zzyzzzz.

```
# Crunch 4 4 -f unicode_test.lst japanese -t @@@% -l @xdd
```

Crunch will load some **Japanese** characters from the unicode_test character set file. The output will start at @日00 and end at @語99.

Hope you enjoyed our article and details.